САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Фронт-энд разработка

Отчет

Лабораторная работа №2

Выполнил:
Алёшкин Дмитрий

Группа:
J3211

Проверил:
Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Реализовать моковое API средствами JSON-сервера и подключить к нему авторизацию, вывод актуальной информации о курсах доллара и евро, а также текущих расходов.

Ход работы

В данной работе изначально было реализовано получение актуального курса доллара и евро. Решено было выводить эту информацию в navbar.

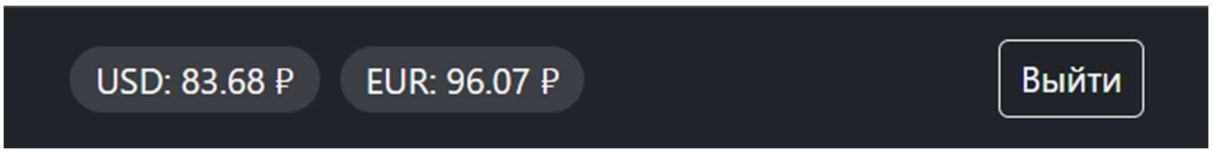

Рис. 1. Вывод актуального курса валют в navbar

Далее была реализована как логика регистрации, так и входа. Данный код можно посмотреть в приложении 1. Все новые данные, а также данные уже зарегистрированных пользователей, хранятся в фале db.json. Все дальнейшие динамические изменения на сайте, влияющие на данные, так же будут изменять именно данный файл.

Было решено сделать демонстрационный вариант сайта, который изначально видит пользователь без регистрации. Данный код находится в файле api.js.

В личном кабинете пользователя теперь есть возможность добавлять, изменять и удалять категории. Они отображаются в формате от большего лимита к меньшему.

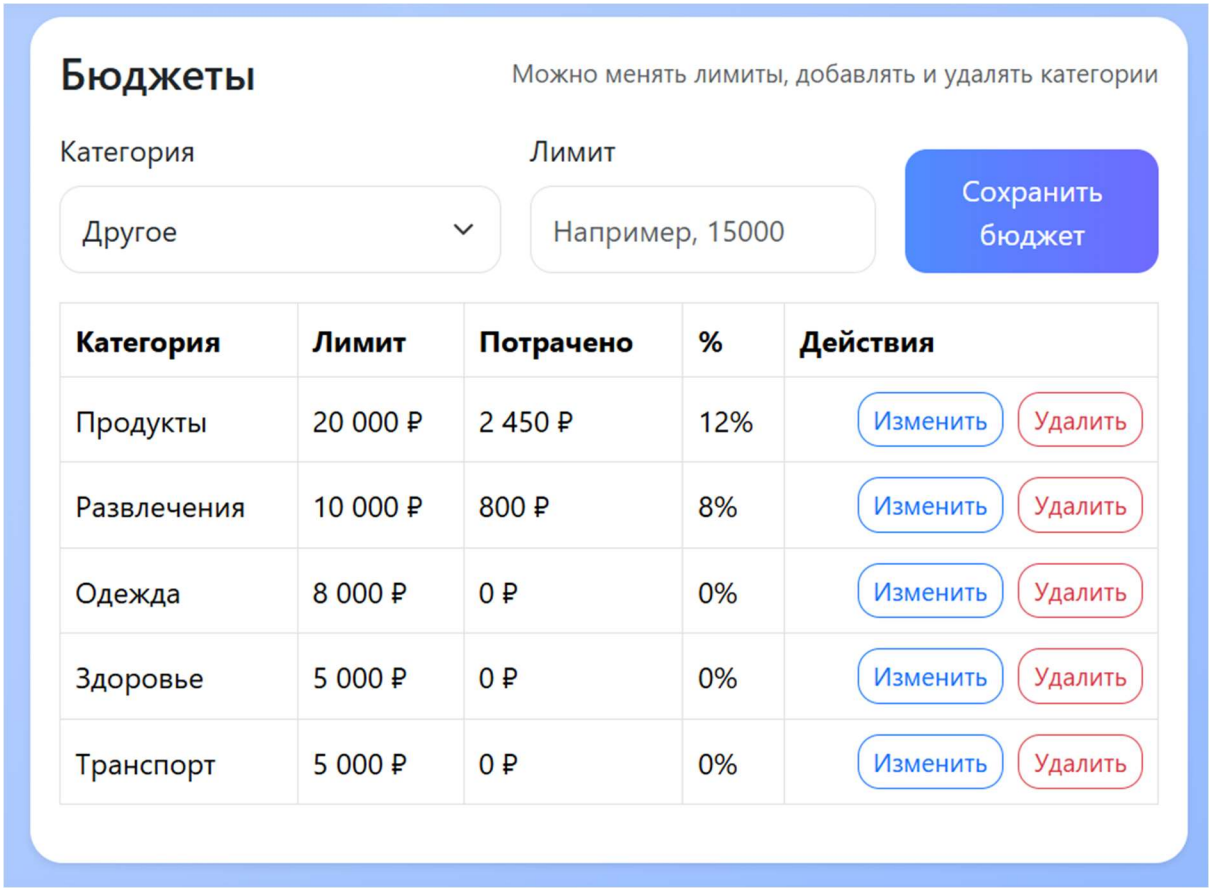

Рис. 2. Пример распределения бюджета по категориям

На данной странице реализована круговая диаграмма динамики расходов по категории, чтобы пользователь наглядно мог видеть, на что именно у него уходит бюджет. При наведении на определённый будет выведена информация о тратах с конкретными значениями.

Рис. 3. Круговая диаграмма расходов

Раздел “Транзакции” был доработан и теперь кнопки фильтрации, добавления каких-либо транзакций и их удаления являются рабочими.

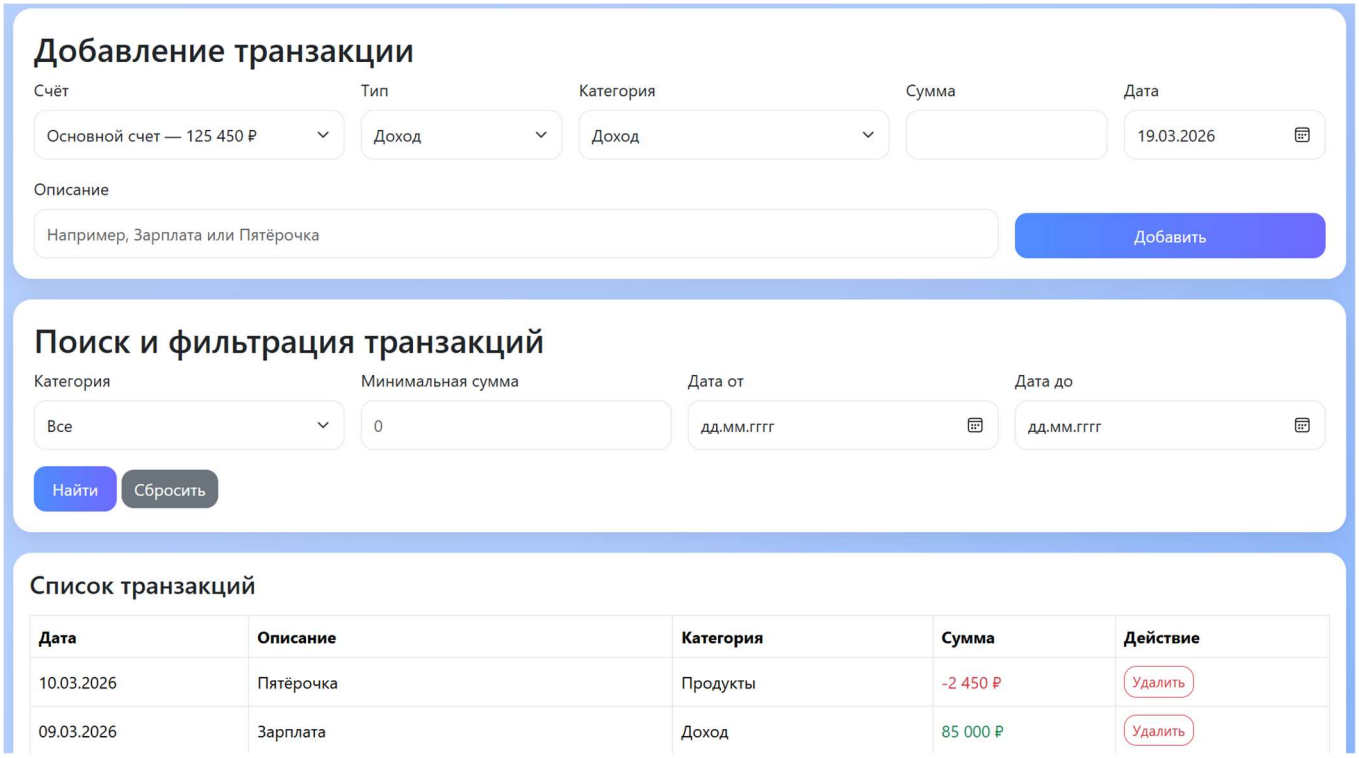

Рис. 4. Реализация функционала страницы “Транзакции”

Стоит отметить, что в разделе “Интеграции” мы можем задавать правила, по которым у нас транзакции будут автоматически отнесены к определённой категории, это работает по описанию самой транзакции. Правила можно добавлять и удалять. Саму реализацию логики в файле api.js можно посмотреть в приложении 2. Подключение реальных банковских карт в данной лабораторной не реализовано.

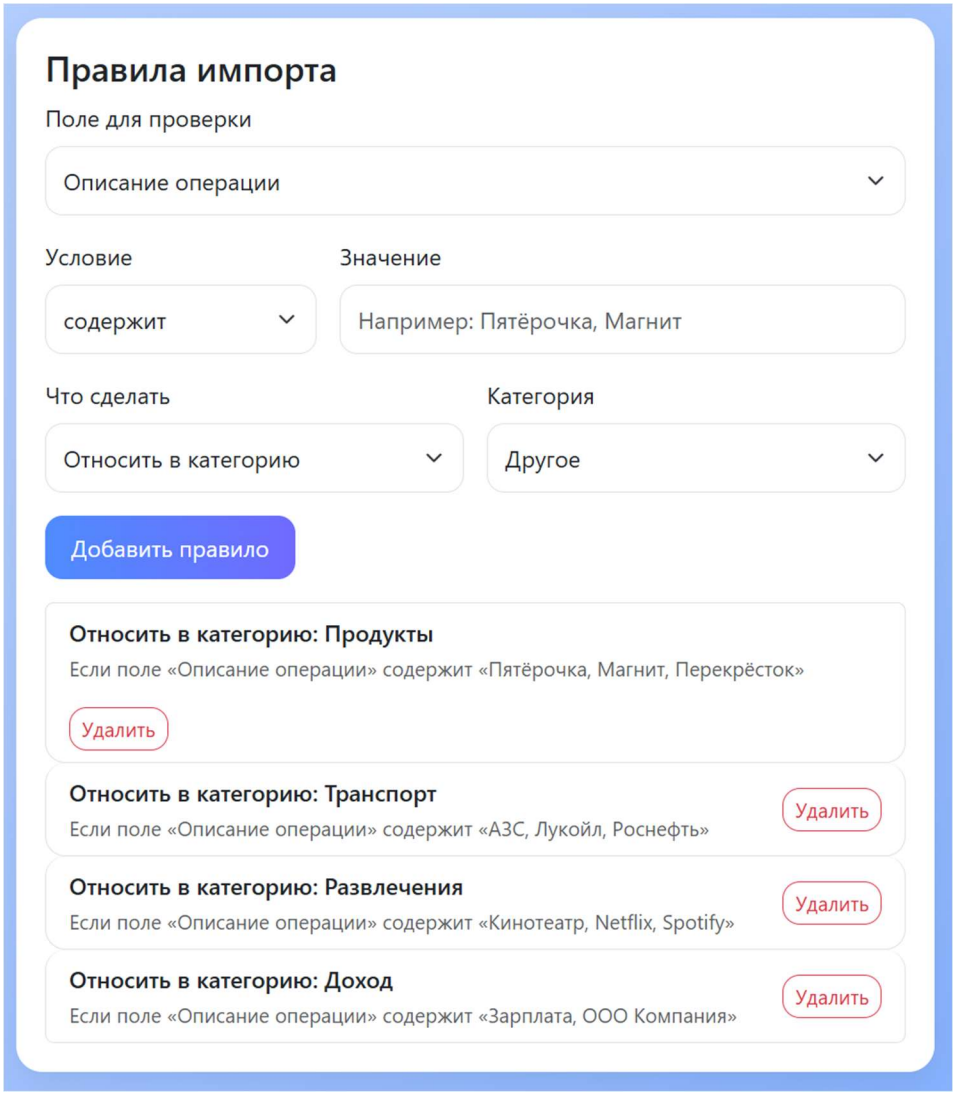

Рис. 5. Пример правил импорта

На странице “Отчёты”, которая автоматически подтягивает все изменения на сайте, можно наглядно увидеть прогнозы по бюджетам, а также динамику расходов и доходов в формате гистограмм.

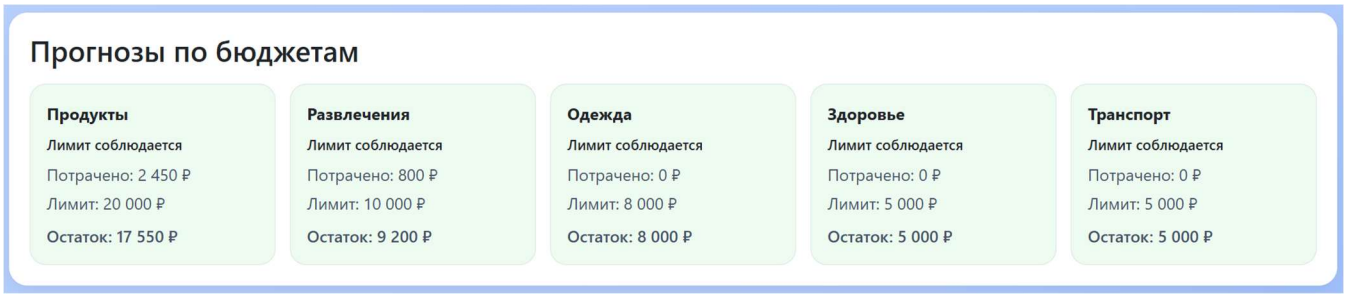

Рис. 6. Пример прогнозов

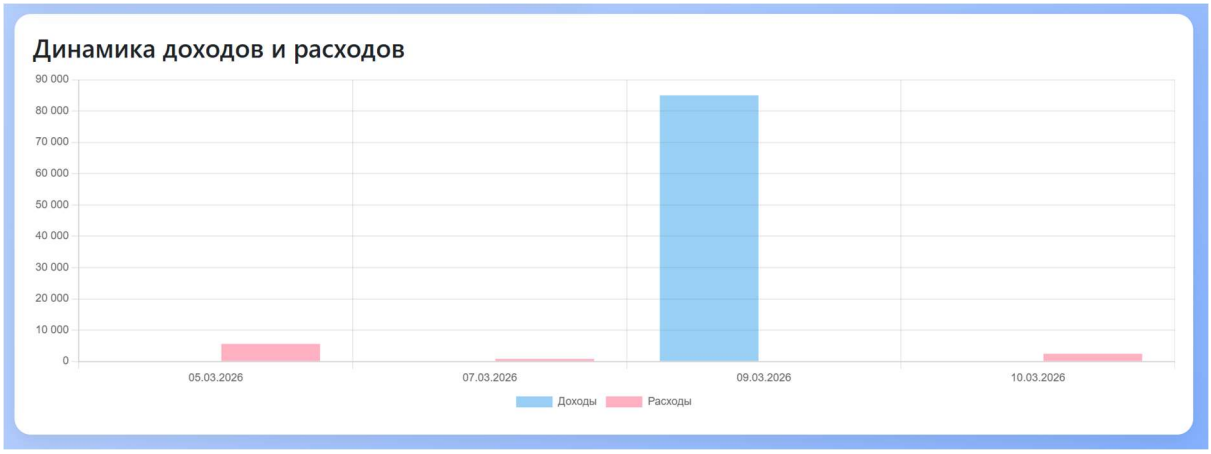

Рис. 7. Пример динамики расходов и доходов

Вывод

Благодаря данной лабораторной работы я разобрался в основах работы с JS и подключению API средствами fetch/axios/xhr. Реализовал моковое API средствами JSON-сервера и подключил к нему авторизацию. Я получил практические навыки создания функционала сайта.

Приложение 1

```
if (loginForm) {
  loginForm.addEventListener('submit', async (event) => {
    event.preventDefault();
    const email = document.getElementById('email').value.trim().toLowerCase();
    const password = document.getElementById('password').value;
    try {
      await Api.login(email, password);
      window.location.href = 'dashboard.html';
    } catch (error) {
      showAlert(normalizeError(error), 'danger');
    }
  });
}
```

Рис. 1. Код реализации логики входа

```
if (registerForm) {
  registerForm.addEventListener('submit', async (event) => {
    event.preventDefault();
    const name = document.getElementById('name').value.trim();
    const email = document.getElementById('email').value.trim().toLowerCase();
    const password = document.getElementById('password').value;
    const password2 = document.getElementById('password2').value;

    if (password !== password2) {
      showAlert('Пароли не совпадают', 'warning');
      return;
    }

    try {
      await Api.register({ name, email, password });
      window.location.href = 'dashboard.html';
    } catch (error) {
      showAlert(normalizeError(error), 'danger');
    }
  });
}
```

Рис. 2. Код реализации логики регистрации

Приложение 2

```
async function applyRulesToTransaction(payload) {
  const rulesRaw = await request('/rules');
  const rules = byUserId(rulesRaw, payload.userId)
    .map(normalizeRule)
    .filter((rule) => rule.active !== false);

  const matchedRule = rules.find((rule) => rule.actionType === 'category'
  if (!matchedRule) return payload;

  return {
    ...payload,
    category: matchedRule.categoryId,
    categoryName: matchedRule.categoryName,
    importRuleId: matchedRule.id,
    importRuleApplied: true,
  };
}
```

Рис. 1. Логика определения к категории и проверка существующих правил